Data Structures and Algorithms (DSA) Assignment

Objective:
Evaluate the candidate’s problem-solving skills, algorithmic thinking, and code efficiency.

Section 1: Arrays and Strings

1. **Find the Missing Number:**
   - Given an array containing `n-1` distinct numbers from `1` to `n`, find the missing number in the sequence.
   - Implement the solution in **O(n)** time complexity.

2. **Longest Substring Without Repeating Characters:**
   - Given a string `s`, find the length of the longest substring without repeating characters.
   - Implement using **Sliding Window technique**.

Section 2: Linked Lists

3. **Detect and Remove Loop in Linked List:**
   - Implement an algorithm to detect if a loop exists in a linked list. If a loop is found, remove it.
   - Use **Floyd’s Cycle Detection Algorithm (Tortoise and Hare method)**.

4. **Reverse a Linked List in Groups of K:**
   - Given a linked list, reverse it in groups of `k`.
   - Implement the solution with **O(n) time complexity**.

Section 3: Trees

5. **Lowest Common Ancestor (LCA) of a Binary Tree:**
   - Given a binary tree and two nodes, find their Lowest Common Ancestor.
   - Implement a recursive solution.

6. **Serialize and Deserialize a Binary Tree:**
   - Design an algorithm to serialize and deserialize a binary tree.
   - Ensure the solution efficiently encodes and decodes the structure of the tree.

Section 4: Graphs

7. **Shortest Path in an Unweighted Graph:**
   - Given an undirected graph and a source node, implement **Breadth-First Search (BFS)** to find the shortest path to all other nodes.

8. **Detect Cycle in a Directed Graph:**
   - Implement an algorithm to detect a cycle in a directed graph using **DFS**.

Section 5: Sorting and Searching

9. **Kth Smallest Element in an Array:**
   - Implement an algorithm to find the `k`th smallest element in an array using **QuickSelect Algorithm**.

10. **Search in a Rotated Sorted Array:**
   - Given a sorted array that has been rotated at some pivot, implement a search function in **O(log n) time complexity**.

Bonus (For Extra Points)

11. **Design a LRU Cache:**
   - Implement a Least Recently Used (LRU) cache using **HashMap and Doubly Linked List**.

12. **Implement Dijkstra's Algorithm:**
   - Given a weighted graph and a source node, implement **Dijkstra's shortest path algorithm**.

Submission Guidelines:
- Submit your code in **GitHub repository** or as a **ZIP file**.
- Code should be properly structured with comments.
- Include a **README file** explaining your approach.

Evaluation Criteria:
- Correctness and Efficiency of Solutions
- Code Readability and Documentation
- Optimization and Edge Cases Handling

Good Luck! 🚀